

Universidade de São Paulo
ICMC - Instituto de Ciências Matemáticas e de Computação

Disciplina: SSC0240 - Bases de Dados
Prof. Dra. Elaine Parros M. de Sousa

CarbonTrack

Sistema de Rastreabilidade para Créditos de Carbono

Membros:

Luiz Gabriel Correia dos Santos - 15639682

Italo Carlos Martins Bresciani - 15461782

Arthur Filliettaz Mendes - 12532055

Thiago Pasquoto Tavares - 15490194

Sumário

1	Introdução	3
1.1	Contextualização	3
1.2	Problema	3
1.3	Objetivos e Usuário-Alvo	3
2	Modelo Entidade-Relacionamento	4
2.1	Descrição geral do sistema e Requisitos de Dados	4
2.2	Principais Funcionalidades	5
2.3	Restrições de Integridade e Notas do Sistema	6
2.3.1	Notas Gerais	6
2.3.2	Notas sobre Ciclos e Regras de Negócio	7
2.4	Diagrama do Modelo Entidade-Relacionamento	7
3	Correções da Parte 1	9
4	Modelo Relacional	10
4.1	Esquema Relacional	10
4.2	Justificativas para as Decisões de Mapeamento	12
4.3	Notas do Modelo Relacional	15
5	Correções da Parte 2	17
6	Implementação (Parte 3)	18
6.1	Ambiente de Desenvolvimento e Tecnologias	19
6.2	Requisitos de Sistema	19
6.3	Operações de Manipulação de Dados e Regras Automáticas	20
6.3.1	Cadastro de Projeto (INSERT em PROJETO)	20
6.3.2	Cadastro de Atividade (INSERT em ATIVIDADE)	20
6.3.3	Regras de Negócio Automáticas (<i>Triggers</i>)	21
6.4	Consultas SQL	21
6.4.1	C1: Rastreabilidade Completa de Lote (Múltiplas Junções)	21
6.4.2	C2: Desempenho de Projetos (Agrupamento e Agregação)	22
6.4.3	C3: Atividades Finalizadas sem Laudo (Subconsulta Não-Correlacionada)	23
6.4.4	C4: Evolução de Preço de um Lote (Função de Janela)	23
6.4.5	C5: Divisão Relacional (Originadores aprovados por TODAS as Certificadoras)	24
6.4.6	C6: Proprietário Atual de Cada Lote Disponível (Subconsulta Cor- relacionada)	24
6.5	A Aplicação (Protótipo Desktop)	25

6.5.1	Execução Parametrizada das Consultas	25
6.5.2	Funcionalidades de Inserção	25
6.5.3	Funcionalidades de Consulta	26
7	Conclusão	27

1 Introdução

1.1 Contextualização

A mitigação das mudanças climáticas tornou-se uma prioridade global impulsionada pela Agenda 2030 da ONU e pelos Objetivos de Desenvolvimento Sustentável (ODS). Nesse cenário, o mercado de créditos de carbono surge como um mecanismo financeiro essencial. Um crédito de carbono representa a certificação de que uma tonelada de dióxido de carbono (CO₂) equivalente¹ deixou de ser emitida ou foi removida da atmosfera. Para que esse mercado funcione, é vital a atuação de diferentes instituições: **Originadores** (que criam os **Projetos** de redução), **Auditores** (que verificam a execução técnica), **Certificadoras** (que emitem os créditos oficiais) e os **Compradores** (empresas que adquirem os créditos para compensar suas próprias emissões).

1.2 Problema

O principal desafio no mercado de créditos de carbono é garantir a transparência, a rastreabilidade e a integridade dos ativos gerados, evitando fraudes como a dupla contagem (vender o mesmo crédito duas vezes) ou a emissão de créditos sem lastro real. Atualmente, a descentralização das informações dificulta a auditoria e a verificação do ciclo de vida completo de um crédito, desde a **Atividade** de campo que o gerou até a sua negociação e eventual aposentadoria no mercado financeiro². É necessário um sistema rigoroso de gestão de dados capaz de mapear e validar cada etapa desse fluxo.

1.3 Objetivos e Usuário-Alvo

O sistema CarbonTrack tem como propósito principal atuar como uma plataforma de base de dados robusta para registrar, rastrear e regular o ciclo de vida e a comercialização de créditos de carbono. Seus objetivos incluem: garantir a integridade referencial entre **Atividades** de campo e os **Lotes** de créditos gerados, mapear as transferências de titularidade e fornecer um histórico confiável de precificação.

O usuário-alvo abrange todos os atores corporativos e institucionais do mercado de carbono: empresas desenvolvedoras de **Projetos (Originadores)**, entidades de auditoria independente (**Auditores**), órgãos certificadores internacionais (**Certificadores**) e empresas **Compradoras** de créditos para compensação ambiental.

¹O CO₂ equivalente (CO₂e) é uma métrica universal utilizada para comparar as emissões de vários gases de efeito estufa (como metano e óxido nitroso) com base em seu Potencial de Aquecimento Global (GWP).

²A "aposentadoria" (*retirement*) de um crédito de carbono ocorre quando o **Comprador** final o utiliza para compensar suas emissões. O crédito é então retirado permanentemente dos registros públicos para garantir que não volte a circular no mercado.

2 Modelo Entidade-Relacionamento

2.1 Descrição geral do sistema e Requisitos de Dados

O fluxo de dados da plataforma CarbonTrack inicia-se com o cadastro das instituições participantes, modeladas pela entidade **Pessoa Jurídica**, identificada unicamente pelo seu CNPJ (identificador único universal) e descrita por sua Razão Social (dado cadastral legal), Nome Fantasia (dado cadastral legal), Status (indica se a instituição está apta a operar) e um Endereço (atributo que é composto por CEP, Estado e Rua; modelado para permitir o armazenamento simultâneo dos endereços de sede e filiais de uma mesma empresa). Toda instituição no sistema deve ser obrigatoriamente um **Agente de Mercado** ou um **Agente de Conformidade**, garantindo a separação entre quem lucra com os créditos e quem os fiscaliza.

O **Agente de Mercado** é categorizado por um atributo de Tipo (diferencia regras gerais de mercado), podendo atuar como **Originador** (possuindo Setor de Atuação e um documento de Capacidade Técnica) e **Comprador** (possuindo Perfil Comprador) simultaneamente. Já o **Agente de Conformidade** possui uma Atribuição e pode atuar como um **Auditor** (identificado por seu Registro de Acreditação e respectiva Data de Validade da Acreditação) ou um **Certificador** (descrito pelo Padrão de Certificação que segue, como o padrão Verra³).

No escopo operacional, o **Originador** realiza **Atividades** de campo (como plantio ou monitoramento), identificadas pelo Código de Ordem de Serviço (identificador no contexto do projeto), possuindo Descrição da Atividade, Custo Operacional (dados monetários de execução), cronograma com Data de Início e Data de Fim, gerando a Duração (característica derivada das datas balizadoras) e um Crédito Estimado (projeção do volume de carbono). Várias **Atividades** podem estar agrupadas dentro de um **Projeto** de mitigação ambiental, o qual é unicamente identificado pelo Número de Licença Ambiental, possuindo Nome do Projeto, Metodologia Aplicada, cronograma próprio de Data de Início e Data de Fim, além de sua Duração. É mandatório que um **Projeto** contenha **Atividades** para existir no sistema.

A regulação exige que o **Auditor** realize inspeções nas **Atividades**, originando um **Laud** (identificado por Número do Protocolo da Auditoria, com Parecer Final, Crédito Real aferido, e uma URL do Documento apontando para o arquivo original). Posteriormente, o **Certificador** analisa o **Laud** e **Verifica** a **Atividade**, que por sua vez **Gera** um **Lote** de créditos de carbono. O **Lote** é a representação do ativo financeiro, identificado por Número de Série de Registro (identificador único global), detalhando a Quantidade de Créditos (toneladas de CO2 equivalentes), o Ano de Geração, o Valor estipulado e seu Status do Ciclo de Vida (estado vitalício restrito a Disponível, Aposentado

³A Verra gerencia o *Verified Carbon Standard* (VCS), que é um dos maiores e mais rigorosos programas de certificação de redução de emissões no mercado voluntário global.

ou Inativo). Imediatamente após ser gerado, a posse inicial do **Lote** é inferida indiretamente ao **Originador** que realizou a respectiva atividade.

Para acompanhar a volatilidade do mercado, cada **Lote** possui um **Histórico de Preços**, identificada parcialmente pela Data e completamente observando também o Número de série do Lote, armazenando o Preço ao longo do tempo. Por fim, a transferência de titularidade ocorre por meio de uma **Transação**, que conecta o **Agente de Mercado** (em papéis de comprador e vendedor) ao **Lote**. A **Transação** é identificada pela Nota Fiscal, contendo Data e Hora e o Valor da negociação.

2.2 Principais Funcionalidades

O sistema foi projetado para suprir operações interativas por parte de seus usuários, contemplando cadastros (inserções), atualizações e remoções (físicas e lógicas):

- **Originador:**

- Cadastra novos **Projetos** de mitigação ambiental.
- Registra **Atividades** de campo, vinculando-as (ou não) a um projeto.
- Visualiza os **Lotes** gerados e atribuídos à sua empresa após certificação.
- Registra **Transações** de venda de Lotes Disponíveis para outros agentes.
- Exclui cadastros de Projetos ou **Atividades** inseridos erroneamente no sistema, desde que ainda não tenham passado por auditoria formal.

- **Comprador:**

- Consulta histórico e origem de **Lotes** de carbono antes da aquisição.
- Participa de **Transações** de compra de créditos de carbono.
- Solicita a alteração do Status do Ciclo de Vida de um Lote para "Aposentado" visando a compensação de suas pegadas de carbono (atualização).
- Vende ativos que estão sob sua posse e pode realizar operações de mudança de valor dos lotes que possui.

- **Auditor:**

- Consulta as **Atividades** de campo ativas no sistema.
- Registra **Laudos** técnicos baseados na vistoria técnica destas atividades, inserindo pareceres e URLs de documentos.
- Remove rascunhos de Laudos contendo erros de preenchimento, antes do envio definitivo para a análise da certificadora.

- **Certificador:**

- Analisa os **Laudos** submetidos pelos auditores.
- Emite e registra novos **Lotes** de carbono baseados em laudos com pareceres positivos.
- Invalida (remoção lógica/inativação) Lotes já emitidos caso sejam detectadas fraudes ou dupla contagem em revisões posteriores, garantindo o *compliance* e a integridade da plataforma.

A partir dessas operações o banco de dados deve suportar buscas gerenciais complexas, garantindo consultas estratégicas e analíticas, tais como:

1. **Rastreabilidade de Lote:** Dado o número de série de um **Lote**, retornar o histórico completo, desde a **Transação** de compra até o **Projeto/Atividade** que o gerou, incluindo o **Certificador** e **Auditor** envolvidos (Múltiplas Junções).
2. **Desempenho de Projetos:** Listar o total de créditos gerados e o valor financeiro movimentado agrupado por **Projeto** (Agrupamento e Funções de Agregação).
3. **Auditoria de Conformidade:** Identificar **Atividades** finalizadas que ainda não possuem um **Laudos** emitido por um **Auditor**.
4. **Análise de Flutuação de Preço:** Recuperar a evolução histórica de preços do crédito de um **Lote** específico ao longo do ano.
5. **Filtro de Certificação:** Encontrar os **Originadores** que possuem **Atividades** aprovadas por *todas* as **Certificadoras** ativas no sistema, demonstrando alto nível de adequação e conformidade global.

2.3 Restrições de Integridade e Notas do Sistema

Para assegurar a consistência dos dados e refletir as regras de negócio do mercado de carbono, foram estabelecidas as seguintes notas:

2.3.1 Notas Gerais

- **Unicidade e Dependência:** O CNPJ deve ser único para qualquer **Pessoa Jurídica**. O **Histórico de Preços** é dependente do **Lote**.
- **Separação de Papéis (Disjunção):** Um **Agente de Mercado** jamais pode auditar ou certificar créditos, impedindo conflitos de interesse.
- **Participação Total:** Um **Projeto** não pode existir sem estar associado a, pelo menos, uma **Atividade**.

- **Nota 1 (N1) - Endereço Multivalorado:** Permite armazenar endereços de sede e filiais.
- **Nota 2 (N2) - Entidade Projeto:** Um projeto representa um conjunto de atividades relacionadas à geração de créditos ambientais.
- **Nota 3 (N3) - Detentor Inicial:** O Lote deve ser atribuído inicialmente ao Originador da atividade.
- **Nota 5 (N5) - Ciclo de Vida do Ativo:** Status restrito a "Disponível", "Aposentado" e "Invalidado".
- **Nota 6 (N6) - Histórico de Preço:** Relação 1:N para guardar atualizações cronológicas de valor.

2.3.2 Notas sobre Ciclos e Regras de Negócio

- **Nota 4 (N4) - Automação de Posse Inicial:** O sistema implementará uma regra de automação para que, no momento em que a Certificadora **Verifica** uma Atividade e esta **Gera** um respectivo Lote, o sistema identifique o dono inicial consultando o Originador responsável por **Realizar** essa Atividade.
- **Nota 7 (N7) - Ciclo da Negociação:** Um agente não pode comprar um lote que já está sob sua posse. Como esta restrição de ciclo não pode ser garantida estritamente pelas cardinalidades do MER, ela será tratada como uma regra de validação na camada de aplicação.

2.4 Diagrama do Modelo Entidade-Relacionamento

Abaixo está o diagrama do Modelo Entidade-Relacionamento (MER). Notas visuais foram removidas da imagem para maior clareza, estando agora descritas textualmente.

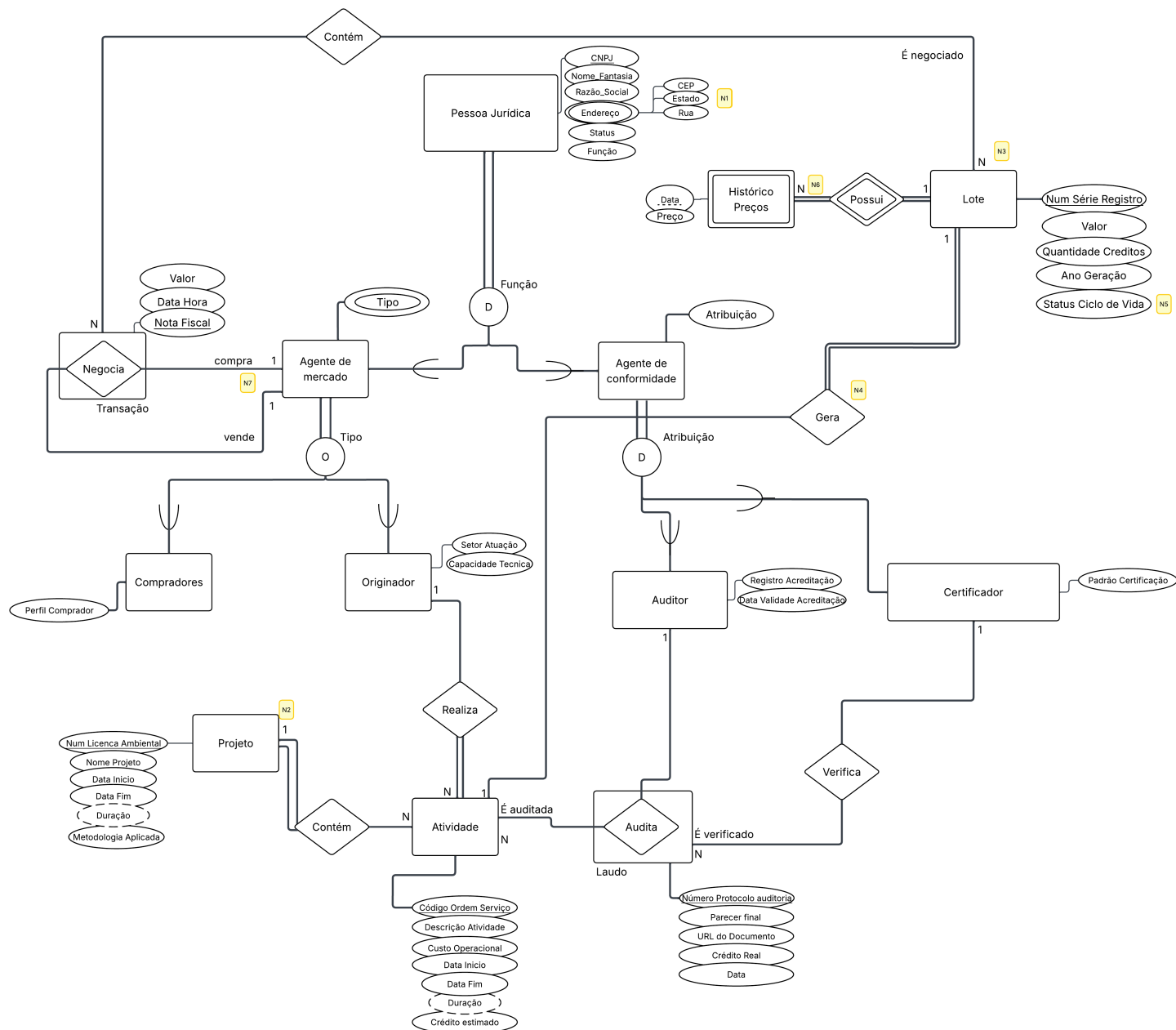


Figura 1: Diagrama MER do projeto CarbonTrack.

3 Correções da Parte 1

Antes da realização da segunda etapa do projeto, foram efetuadas modificações no modelo original visando corrigir inconsistências e otimizar a estrutura relacional futura. As alterações são detalhadas a seguir:

- **Redução de Termos Técnicos na Descrição Geral:** Visando facilitar a compreensão do sistema por stakeholders e clientes não técnicos, os requisitos de dados (seção 2.1) foram revisados para remover jargões excessivos, tornando a descrição funcional mais acessível.
- **Adição da Nota N7 sobre Ciclo da Negociação:** Foi identificada a necessidade de impedir que um Agente de Mercado adquira créditos de sua própria titularidade. Como restrições de ciclo complexas possuem limitações de representação no MER, adicionou-se a Nota 7 para garantir que essa validação ocorra na camada de lógica do sistema.
- **Reestruturação da Origem de Lotes (CR Gera):** Na versão anterior, a conexão direta entre **Originador** e **Lote** causava perda de rastreabilidade sobre qual processo produtivo gerou o ativo. O modelo foi corrigido adicionando-se um Relacionamento **Gera** entre **Atividade** e **Lote**. Agora, o Originador é rastreado indiretamente via Atividade, preservando o histórico técnico.
- **Detalhamento da Automação (N4):** A Nota 4 foi expandida para descrever o processo de inferência de posse. Isso esclarece como o banco de dados deve se comportar no momento da emissão do crédito, garantindo que o vínculo entre o executor da atividade e o primeiro dono do lote seja automático e íntegro.
- **Organização de Notas sobre Ciclos:** Para melhorar a legibilidade das restrições, as notas que abordam consistência de fluxos (como N4 e N7) foram movidas para uma subseção dedicada, facilitando a consulta por parte dos desenvolvedores.
- **Quebra do Relacionamento Ternário "Negocia":** O relacionamento ternário original entre Comprador, Vendedor e Lote foi substituído por uma **Agregação** chamada **Transação**. A Transação agora se relaciona com o Lote através do CR **Contém**. Esta mudança permite que uma única Nota Fiscal englobe múltiplos Lotes (N:N), assim como Lotes podem se envolver em mais de uma transação durante o ciclo de vida do sistema, e trata a venda como um evento fiscal independente, reduzindo redundâncias.
- **Decisão Arquitetural sobre a Identificação do Proprietário Atual do Lote:** Foi cogitada uma mudança no MER para tratar a identificação explícita do proprietário atual do **Lote**, como a criação de um relacionamento **Possui** (1:N) entre

Agente de Mercado e Lote. Contudo, optou-se por não introduzir essa redundância no modelo, pois ela geraria ciclos e representaria uma potencial vulnerabilidade à inconsistência de dados caso as atualizações não fossem devidamente sincronizadas. Assim, a identificação do proprietário atual é obtida por inferência a partir de dois caminhos já existentes: caso o Lote possua negociações registradas, o proprietário corresponde ao comprador da transação mais recente; caso contrário, o proprietário é o **Originador** da **Atividade** que gerou o Lote. A principal desvantagem assumida com essa abordagem é o custo computacional de realizar essa consulta com frequência em um cenário de alto volume de transações.

- **Melhorias de Apresentação:** O diagrama MER foi limpo, removendo-se balões de notas que poluíam a visualização. Além disso, a página foi configurada em modo paisagem (*landscape*) para permitir a visualização de todos os atributos e entidades em escala adequada.

4 Modelo Relacional

4.1 Esquema Relacional

Abaixo está o diagrama do Modelo Relacional do sistema CarbonTrack, obtido a partir do mapeamento sistemático do Modelo Entidade-Relacionamento apresentado na seção anterior. O esquema reflete as decisões de mapeamento detalhadas nas subseções seguintes, com destaque para o tratamento das hierarquias de especialização, dos atributos multivalorados e dos relacionamentos com cardinalidades distintas.

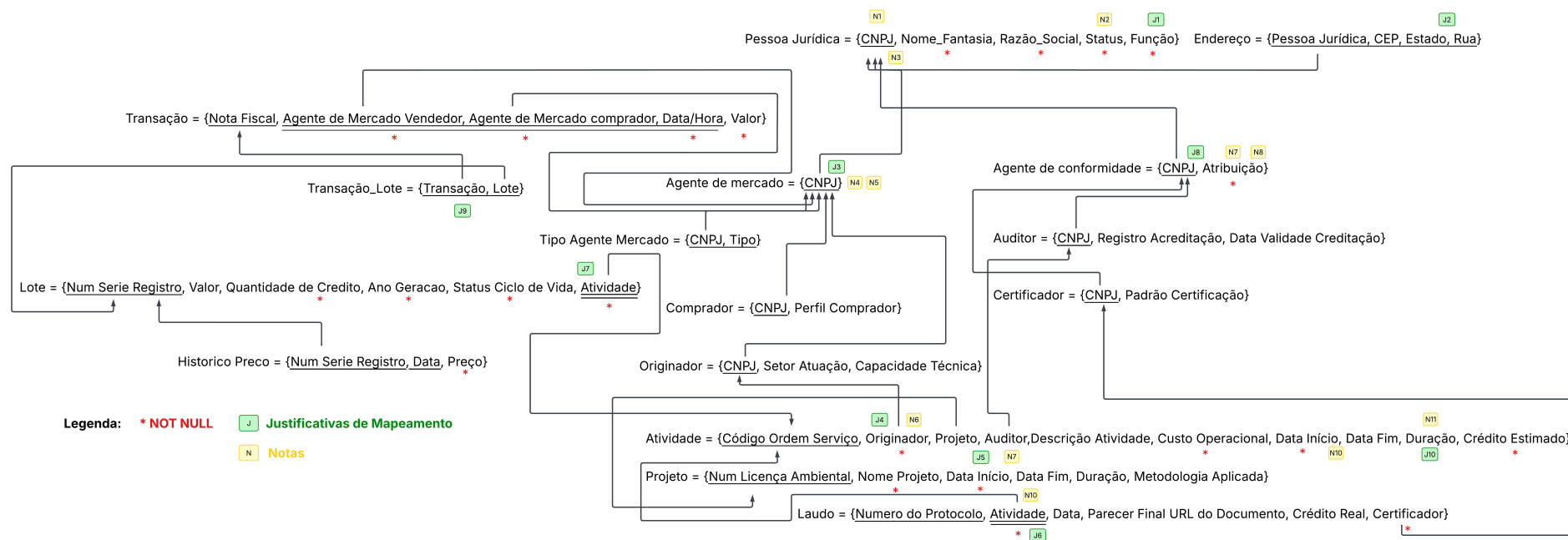


Figura 2: Diagrama do Modelo Relacional do projeto CarbonTrack.

4.2 Justificativas para as Decisões de Mapeamento

- **Justificativa 1 (J1): Estrutura de Pessoa Jurídica:** Optamos pela criação de uma tabela **Pessoa Jurídica** e uma tabela para cada subclasse dela, sendo o **CNPJ** a chave estrangeira (FK) das tabelas da subclasse. Duas alternativas foram consideradas e descartadas. Mapear apenas as subclasses eliminaria o custo de *JOINS*, porém, para comportar o atributo multivalorado **ENDEREÇO**, seria necessário criar duas tabelas independentes de endereço (uma referenciando **Agente de Mercado** e outra referenciando **Agente de Conformidade**), gerando redundância estrutural e dificultando consultas globais sobre todas as instituições cadastradas. Por outro lado, criar uma única tabela "*Pezão*" com discriminador (ex.: **Tipo_PJ**) resultaria em muitos valores NULL decorrentes dos atributos exclusivos de cada subclasse, além de gerar confusão semântica, uma vez que a regra de negócio separa estritamente os papéis de conformidade e de mercado.
- **Justificativa 2 (J2): Atributo Endereço:** O atributo **Endereço** é composto e multivalorado, pois uma mesma Pessoa Jurídica pode possuir endereços de sede e de filiais simultaneamente. A solução estruturalmente correta escolhida é criar uma nova relação **ENDEREÇO** que recebe os componentes simples do endereço (logradouro, estado e CEP) junto à chave primária da relação proprietária (**CNPJ**). A chave primária desta nova tabela é a combinação (**CNPJ**, **CEP**, **Estado**, **Rua**), garantindo que o mesmo endereço não seja duplicado para uma mesma empresa, ao mesmo tempo que permite múltiplas entradas para empresas com mais de uma localização. A alternativa de embutir colunas fixas de endereço diretamente em **PESSOA_JURIDICA** (ex.: **Endereco1_CEP**, **Endereco2_CEP**) foi descartada por limitar artificialmente o número de endereços e gerar valores NULL para empresas com apenas uma sede.
- **Justificativa 3 (J3): Especialização de Agente de Mercado:** Optou-se por não incluir o atributo **Tipo** diretamente na superclasse **Agente de Mercado**. Em vez disso, criou-se uma tabela auxiliar **TIPO_AGENTE_MERCADO** para registrar a quais subtipos cada agente pertence, além de tabelas distintas para cada subclasse (**Originador** e **Comprador**). Como a especialização é **sobreposta** (um mesmo agente pode ser simultaneamente Originador e Comprador), essa abordagem permite identificar rapidamente todos os papéis de um agente apenas utilizando junções, sem a necessidade de varrer múltiplas tabelas de subclasse. A principal desvantagem é a fragmentação estrutural: para recuperar o perfil completo, o SGBD precisará executar *JOINS* cruzando **PESSOA_JURIDICA**, **AGENTE_MERCADO**, **TIPO_AGENTE_MERCADO** e as tabelas de subclasse. Ressalta-se ainda que essa abordagem não garante nativamente a especialização total via DDL simples, exigindo validação complementar na camada de aplicação.

- **Justificativa 4 (J4): Relacionamento Realiza (1:N entre Originador e Atividade):** O relacionamento **Realiza** foi mapeado pela Técnica de Chave Estrangeira: a tabela do lado "N"(ATIVIDADE) absorveu a chave primária do lado "1"(ORIGINADOR) como FK. Essa escolha mantém o banco otimizado, atendendo plenamente ao requisito de que cada Atividade está vinculada **obrigatoriamente** a um Originador, o que garante a **totalidade**. A alternativa de Referência Cruzada (criação de uma tabela de ligação, como REALIZA_ATIVIDADE) é desencorajada para relacionamentos 1:N, pois introduz uma relação extra desnecessária, obrigando o SGBD a executar uma operação adicional de *EQUIJUNÇÃO* apenas para descobrir o responsável pela atividade.
- **Justificativa 5 (J5): Mapeamento Múltiplo de Relacionamentos 1:N na Tabela Atividade:** Em vez de criar tabelas separadas para os relacionamentos **Realiza**, **Contém** e **Audita**, optou-se pela Técnica de Chave Estrangeira em todos os casos. A tabela ATIVIDADE, que representa o lado "N" de todos esses relacionamentos, absorveu as chaves estrangeiras do **Originador**, do **Projeto** e do **Auditor**. As justificativas para essa escolha espelham o raciocínio da J4, já que uma instância de **Projeto** precisa estar associada a uma **Atividade**. Porém, há um agravante: a criação de tabelas de ligação para cada relacionamento 1:N (como REALIZA_ATIVIDADE, CONTEM_ATIVIDADE e AUDITA_ATIVIDADE) não apenas introduziria relações extras, mas também fragmentaria a estrutura do banco, tornando as consultas mais complexas e menos performáticas.
- **Justificativa 6 (J6): Mapeamento da Agregação Laudo:** Como o relacionamento **Audita** (entre Auditor e Atividade) foi embutido diretamente na tabela ATIVIDADE via FK do Auditor (conforme J5), a tupla de Atividade passou a representar, por si só, o próprio relacionamento gerador da agregação. Bastou, portanto, inserir a FK da Atividade dentro da tabela LAUDO para mapear a agregação de forma completa: o Laudo sabe exatamente qual Atividade auditou e, por transição estrutural, sabe quem é o Auditor, sem necessidade de armazenar novamente o CNPJ do Auditor no Laudo. A alternativa de incluir também a FK do Auditor diretamente em LAUDO foi descartada por introduzir redundância e risco de inconsistência: os dois campos deveriam sempre apontar para o mesmo Auditor da Atividade, o que não pode ser garantido nativamente pelo DDL.
- **Justificativa 7 (J7): Relacionamento Gera (1:1 entre Atividade e Lote):** O relacionamento **Gera** foi mapeado pela Técnica de Chave Estrangeira, com a FK posicionada na tabela LOTE (e não em ATIVIDADE), com as restrições UNIQUE e NOT NULL. Essa posição é semanticamente mais precisa: todo Lote necessariamente deriva de uma Atividade, enquanto uma Atividade pode ainda não ter gerado nenhum Lote — o que tornaria inevitável a presença de valores NULL caso a FK estivesse no lado

oposto. A restrição **UNIQUE** é o elemento central do mapeamento, pois transforma uma FK comum (que naturalmente expressaria cardinalidade 1:N) em uma FK que impõe o 1:1 diretamente no DDL, sem necessidade de *triggers* ou lógica adicional de aplicação. A criação de uma tabela intermediária de ligação foi descartada pelos mesmos motivos da J4.

- **Justificativa 8 (J8): Especialização de Agente de Conformidade:** Optou-se por criar uma tabela **AGENTE_CONFORMIDADE** para a superclasse e tabelas distintas para cada subclasse (**AUDITOR** e **CERTIFICADOR**), com seus atributos específicos. O CNPJ é a chave primária de todas as tabelas, e a FK das subclasses referencia sempre **AGENTE_CONFORMIDADE**. O atributo **Atribuição**, presente na superclasse, atua como discriminador do tipo do agente, permitindo identificar a subclasse correspondente sem consultar as tabelas especializadas diretamente. Diferentemente do mapeamento adotado para **Agente de Mercado** (J3), não foi criada uma tabela auxiliar de tipo, pois a especialização aqui é **disjunta**: cada agente pertence a exatamente uma subclasse, tornando desnecessário o controle de múltiplos papéis simultâneos. A desvantagem dessa abordagem é que, para recuperar o perfil completo de um agente de conformidade, o SGBD precisará cruzar **AGENTE_CONFORMIDADE** com a tabela da subclasse correspondente. O modelo relacional sozinho não garante nativamente nem a disjunção nem a totalidade da especialização, conforme tratado nas Notas N8 e N9.
- **Justificativa 9 (J9): Relacionamento N:N entre Transação e Lote:** O relacionamento entre **Transação** e **Lote** foi mapeado via tabela intermediária **TRANSACAO_LOTE**, devido à sua natureza Muitos-para-Muitos. Um Lote pode ser objeto de múltiplas transações ao longo de seu ciclo de vida (vendas sucessivas entre agentes), e uma única Transação (nota fiscal) pode englobar um conjunto de diferentes Lotes.

Duas alternativas foram consideradas e descartadas. A primeira seria inserir uma FK de Transação diretamente na tabela **LOTE**: isso limitaria cada Lote a uma única transação, tornando impossível registrar o histórico de vendas sucessivas do ativo e destruindo a rastreabilidade do ciclo de vida. A segunda alternativa seria inserir uma FK de Lote diretamente na tabela **TRANSACAO**: isso restringiria cada nota fiscal a um único Lote, impedindo que uma negociação envolva múltiplos ativos simultaneamente, o que contraria diretamente a regra de negócio. A criação da tabela intermediária é, portanto, a única solução estruturalmente correta para essa cardinalidade, pois preserva ambas as direções do relacionamento sem redundância.
- **Justificativa 10 (J10): Atributo Derivado Duração:** Optou-se por manter o atributo **Duração** fisicamente nas tabelas **PROJETO** e **ATIVIDADE**, ainda que seja derivado das datas de início e fim.

Duas alternativas foram consideradas. A primeira seria **não armazenar o atributo**, calculando-o sempre em tempo de execução via expressão SQL (`Data_Fim - Data_Inicio`) a cada consulta. Essa abordagem elimina qualquer risco de inconsistência, mas impõe um custo computacional recorrente em consultas de auditoria que varrem grandes volumes de registros, exatamente o caso de uso mais frequente do sistema. A segunda alternativa seria utilizar uma **coluna gerada** (`GENERATED ALWAYS AS`), recurso suportado pelo PostgreSQL, que manteria o valor atualizado automaticamente pelo SGBD sem intervenção de *triggers* ou aplicação. Embora tecnicamente elegante, essa solução introduz dependência de uma funcionalidade específica do SGBD, reduzindo a portabilidade do esquema.

Optou-se pela persistência física do atributo por oferecer o melhor equilíbrio entre desempenho de leitura e simplicidade de implementação, dado que a *Duração* é determinada no cadastro e permanece estável enquanto as datas não forem alteradas. A consistência do valor é garantida por *triggers* ou pela camada de aplicação, conforme detalhado na Nota N11.

4.3 Notas do Modelo Relacional

As notas a seguir documentam restrições de implementação, decisões de integridade referencial e aspectos que exigem controle complementar na camada de aplicação ou via *triggers*, uma vez que não podem ser garantidos exclusivamente pelo DDL do modelo relacional.

- **Nota 1 (N1) – Integridade Referencial:** Os CNPJs nas tabelas de subclasse e na tabela `ENDEREÇO` são Chaves Estrangeiras (FK) referenciando `PESSOA_JURIDICA`. Aplica-se `ON DELETE CASCADE` para que a exclusão de uma Pessoa Jurídica remova automaticamente seus endereços e registros de especialização, evitando registros órfãos.
- **Nota 2 (N2) – Restrição de Domínio:** O campo `Status` da tabela `PESSOA_JURIDICA` deve ter domínio restrito aos valores `'Apto'` e `'Inapto'`, imposto via cláusula `CHECK` no DDL.
- **Nota 3 (N3) – Especialização:** Para garantir totalidade e disjunção na especialização de **Pessoa Jurídica** em **Agente de Mercado** e **Agente de Conformidade** (visto que o SGBD não impõe isso nativamente), devem ser utilizadas **triggers** ou lógica de aplicação para validar se o CNPJ inserido respeita as regras de negócio.
- **Nota 4 (N4) – Propagação de Exclusão:** Como o CNPJ nas tabelas `ORIGINADOR`, `COMPRADOR` e `TIPO_AGENTE_MERCADO` referencia `AGENTE_MERCADO`, configura-se `ON`

DELETE CASCADE. Dessa forma, a exclusão do Agente central propaga automaticamente para suas categorias e papéis, evitando registros fantasmas no banco de dados.

- **Nota 5 (N5) – Sincronização de Identidade:** O Modelo Relacional sozinho não garante que os valores gravados em TIPO_AGENTE_MERCADO reflitam a real existência do agente nas tabelas ORIGINADOR ou COMPRADOR. É necessário implementar uma camada de aplicação ou *triggers* para assegurar que a inserção em uma das subclasses seja sempre refletida corretamente em TIPO_AGENTE_MERCADO, e vice-versa.
- **Nota 6 (N6) – Integridade Referencial:** Adotando a Técnica de Chave Estrangeira, o campo `Originador` em ATIVIDADE deve apontar obrigatoriamente para um CNPJ existente e válido em ORIGINADOR. Para este relacionamento, a configuração ideal é ON DELETE RESTRICT: se um Originador já realizou atividades no campo, seu cadastro não pode ser excluído do sistema, pois isso destruiria a rastreabilidade do projeto de mitigação ambiental.
- **Nota 7 (N7) – Ciclo de Vida da Auditoria:** Como a Atividade é cadastrada antes de qualquer fiscalização e o Auditor só é vinculado a ela posteriormente, a coluna `Auditor` na tabela ATIVIDADE deve nascer com valor NULL e ser atualizada via UPDATE no momento da contratação. A camada de aplicação é responsável por validar se uma Atividade já possui Auditor preenchido antes de permitir a emissão do respectivo Laudo.
- **Nota 8 (N8) – Garantia de Disjunção:** A disjunção de **Agente de Conformidade** é parcialmente garantida via DDL pela cláusula CHECK sobre o atributo `Atribuição`, restringindo seus valores a 'Auditor' ou 'Certificador'. Para reforçar a consistência, deve-se implementar uma *trigger* BEFORE INSERT nas tabelas AUDITOR e CERTIFICADOR, verificando se o valor de `Atribuição` em AGENTE_CONFORMIDADE corresponde à subclasse sendo inserida e abortando a operação em caso de divergência. Isso impede, por exemplo, que um CNPJ com `Atribuição` = 'Certificador' seja inserido na tabela AUDITOR.
- **Nota 9 (N9) – Garantia de Totalidade:** A totalidade da especialização de **Agente de Conformidade** não é garantida pelo DDL. Para impô-la, a camada de aplicação deve assegurar que toda inserção em AGENTE_CONFORMIDADE seja acompanhada, na mesma transação, da inserção do respectivo registro em AUDITOR ou CERTIFICADOR, conforme o valor de `Atribuição`. Dessa forma, nunca existirá um Agente de Conformidade sem subclasse correspondente.
- **Nota 10 (N10) – Integridade Temporal:** Tanto a tabela PROJETO quanto a tabela ATIVIDADE possuem os atributos `Data_Inicio` e `Data_Fim`. Para garan-

tir que `Data_Fim` nunca seja anterior a `Data_Inicio`, deve-se aplicar a cláusula `CHECK (Data_Fim >= Data_Inicio)` no DDL de ambas as tabelas. Essa restrição é verificada automaticamente pelo SGBD em toda operação de `INSERT` ou `UPDATE`. Recomenda-se, adicionalmente, que a aplicação valide essa regra antes de submeter a operação ao banco, oferecendo uma mensagem de erro mais clara ao usuário.

- **Nota 11 (N11) – Cálculo de Atributo Derivado:** Para garantir a consistência do atributo `Duração` (mapeado fisicamente conforme J10), a implementação deve assegurar que seu valor seja sempre o reflexo fiel da diferença entre `Data_Fim` e `Data_Inicio`. Essa integridade deve ser mantida preferencialmente via **triggers** (`BEFORE INSERT OR UPDATE`) no SGBD, que automatizam o recálculo sempre que houver alteração nas datas, ou por meio de lógica mandatória na camada de aplicação antes da persistência. Isso evita que o banco armazene valores obsoletos ou divergentes das datas balizadoras caso uma atualização ocorra apenas parcialmente.

5 Correções da Parte 2

- **Novas Restrições do Modelo Relacional:** Novas restrições de `NOT NULL` foram adicionadas ao modelo relacional para reforçar a integridade de domínio e garantir que o banco reflita estritamente as regras de negócio do sistema:
 - Na tabela `Projeto`, os atributos `Nome_Projeto` e `Data_Inicio` foram alterados para `NOT NULL`. O nome é essencial para a identificação semântica do projeto em relatórios gerenciais, enquanto a data de início é um marco temporal obrigatório para compor a restrição de consistência cronológica e viabilizar o cálculo do atributo derivado `Duracao`.
 - Na tabela `Transacao`, o atributo `Valor` foi alterado para `NOT NULL`. Sendo o CarbonTrack uma plataforma que lida com ativos financeiros, uma transação sem valor monetário definido fere a integridade dos dados e inviabilizaria as consultas analíticas de faturamento e movimentação de mercado (como a Consulta C2).
 - Na tabela `Pessoa_Juridica`, os atributos base (`Razao_Social`, `Nome_Fantasia` e `Status`) foram alterados para `NOT NULL`. Por se tratar da superclasse estrutural de todas as instituições do sistema, a obrigatoriedade desses dados garante a identidade legal mínima para que qualquer empresa possa operar na plataforma.
 - Na tabela `Agente_Conformidade`, o atributo `Atribuicao` foi alterado para `NOT NULL`. Este atributo atua como o **discriminador** da especialização disjunta.

Sua ausência impediria o sistema de identificar se o agente é um Auditor ou um Certificador, quebrando a lógica de roteamento e validação das tabelas filhas.

- Na tabela **Atividade**, o atributo **Data_Inicio** foi alterado para **NOT NULL**. A definição do início do cronograma operacional é indispensável para gerar o registro de **Duracao** e garantir que o ciclo de vida da geração do crédito de carbono tenha um marco inicial auditável.
 - Na tabela **Lote**, o atributo **Status_Ciclo_de_Vida** foi alterado para **NOT NULL**. O ativo ambiental deve obrigatoriamente estar em um estado conhecido (Disponível, Aposentado ou Invalidado) para que a aplicação possa aplicar as restrições de mercado, como impedir a venda de créditos já aposentados.
 - Na tabela **Laudo**, o atributo **Certificador** foi alterado para **NOT NULL**, garantindo que todo documento de auditoria submetido possua um órgão certificador legalmente responsável por sua futura avaliação.
- **Modificação na Justificativa J2:** A justificativa J2 afirmava que a solução proposta era "a única solução estruturalmente correta". Essa afirmação foi removida uma vez que era drástica e imprecisa.
 - **Modificação na Justificativa J3:** A justificativa J3 afirmava que a solução permitia consultas rápidas sem a necessidade de buscar em outras tabelas. Foi adicionada a especificação sobre a necessidade do uso de junções (JOIN) na consulta.
 - **Modificação na Justificativa J4 e J5:** As justificativas J4 e J5 definiam o requisito sobre cada Atividade estar vinculada a, no máximo, 1 Originador. Como a relação é, na verdade, de totalidade (sempre 1), as justificativas foram modificadas para especificar isso com mais precisão.

6 Implementação (Parte 3)

A terceira etapa do projeto consistiu na materialização do Modelo Relacional especificado na Parte 2 em um banco de dados funcional, no carregamento de dados de teste, no desenvolvimento das consultas previstas e na construção de um protótipo de aplicação que interage com o banco. O foco foi garantir que todo o esquema relacional, incluindo as restrições de integridade e as regras de negócio (Notas N1–N11), fosse fielmente traduzido para comandos SQL e PL/pgSQL.

Todo o código-fonte (scripts SQL, *triggers*, dados e a aplicação) foi versionado em repositório Git e acompanhado de um **README.md** com instruções detalhadas de execução e reprodução do ambiente.

6.1 Ambiente de Desenvolvimento e Tecnologias

O desenvolvimento empregou o seguinte conjunto de ferramentas:

- **PostgreSQL:** SGBD adotado como base do projeto, escolhido pela robustez, ampla aderência ao padrão SQL e suporte nativo a recursos avançados utilizados nas consultas (funções de janela como `LAG()`, `DISTINCT ON` e expressões `CHECK` com operador de regex `~`).
- **SQL e PL/pgSQL:** linguagens do banco, usadas respectivamente para a definição do esquema (DDL), manipulação e consulta de dados (DML/DQL) e para a codificação das *triggers* de regras de negócio.
- **Python 3 (tkinter/ttk + psycopg2):** linguagem e bibliotecas da camada de aplicação. O `tkinter` fornece a interface gráfica desktop nativa e o `psycopg2` é o *driver* de conexão com o PostgreSQL.
- **Docker e Docker Compose:** contêinerização do ambiente, inicializando de forma padronizada e reproduzível os serviços de banco (`postgres_db`) e de administração visual (`pgAdmin`).
- **Git/GitHub:** controle de versão e colaboração entre os membros do grupo.

Além da aplicação, quatro scripts SQL centrais foram desenvolvidos na pasta `scripts_sql/`: `esquema.sql` (DDL: criação das tabelas e restrições), `triggers.sql` (regras de negócio automáticas), `dados.sql` (DML: inserção de dados mockados) e `consultas.sql` (DQL: seis consultas analíticas).

6.2 Requisitos de Sistema

Para reproduzir o ambiente de banco de dados é necessário apenas o **Docker** e o **Docker Compose**. Na raiz do repositório, o comando `docker-compose up -d` cria o contêiner do PostgreSQL (com o banco `CarbonTrack` vinculado ao usuário `rapazinhos`) e o contêiner do `pgAdmin`. Em seguida, os scripts são injetados em ordem (`esquema` → `triggers` → `dados`) via `docker exec`.

Para executar o protótipo gráfico, são requisitos adicionais: **Python 3** com o pacote de sistema `python3-tk` (interface `tkinter`) e a dependência `psycopg2-binary` (listada em `requirements.txt`). A aplicação conecta-se ao banco pela porta 5433 (mapeada do contêiner) usando as credenciais `rapazinhos/1234`.

6.3 Operações de Manipulação de Dados e Regras Automáticas

Esta seção apresenta os trechos de código que implementam as operações de **cadastro** (inserções) e as **regras de negócio** automáticas. Em todas as operações de escrita, os valores fornecidos pelo usuário são enviados ao banco por meio de *placeholders* %s do psycopg2, prática que separa o comando SQL dos dados e elimina o risco de injeção de SQL.

6.3.1 Cadastro de Projeto (INSERT em PROJETO)

Inserir uma nova tupla na tabela PROJETO. Antes da persistência, a aplicação valida o formato das datas (N10) e calcula o atributo derivado Duração em dias (N11). **Localização:** arquivo codigo_sistema-aplicacao/app.py, método CarbonTrackApp.cadastrar_projeto().

```
1 # codigo_sistema-aplicacao/app.py -- CarbonTrackApp.cadastrar_projeto()
2 duracao = (d_fim - d_ini).days if d_fim is not None else None # N11
3 with self.conn.cursor() as cur:
4     cur.execute("""
5         INSERT INTO PROJETO
6             (Num_Licenca_Ambiental, Nome_Projeto, Data_Inicio,
7              Data_Fim, Duracao, Metodologia_Aplicada)
8         VALUES (%s, %s, %s, %s, %s, %s)
9     """, (num, nome, d_ini, d_fim, duracao, metod))
10 self.conn.commit()
```

Código 1: Inserção de Projeto com cálculo da Duração (N11).

6.3.2 Cadastro de Atividade (INSERT em ATIVIDADE)

Inserir uma nova tupla na tabela ATIVIDADE, materializando os relacionamentos 1:N **Realiza** (Originador, obrigatório), **Contém** (Projeto, opcional) e **Audita** (Auditor, opcional). Os CNPJs e o número de licença são resolvidos a partir de caixas de seleção carregadas do próprio banco, garantindo a integridade referencial das FKs antes do envio. **Localização:** codigo_sistema-aplicacao/app.py, método CarbonTrackApp.cadastrar_atividade().

```
1 # codigo_sistema-aplicacao/app.py -- CarbonTrackApp.cadastrar_atividade()
2 with self.conn.cursor() as cur:
3     cur.execute("""
4         INSERT INTO ATIVIDADE
5             (Codigo_Ordem_Servico, Originador, Projeto, Auditor,
6              Descricao_Atividade, Custo_Operacional, Data_Inicio,
7              Data_Fim, Duracao, Credito_Estimado)
8         VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
9     """, (cod, cnpj_orig, num_proj, cnpj_aud, desc, custo,
10          d_ini, d_fim, duracao, cred))
11 self.conn.commit()
```

Código 2: Inserção de Atividade com chaves estrangeiras opcionais.

6.3.3 Regras de Negócio Automáticas (*Triggers*)

As restrições que o DDL não garante sozinho foram implementadas como *triggers* em PL/pgSQL. O Código 3 mostra o gatilho que recalcula automaticamente a **Duração** a cada INSERT/UPDATE (N11), reforçando no banco a mesma regra aplicada na interface.

Localização: scripts_sql/triggers.sql, função calcular_duracao().

```
1  -- scripts_sql/triggers.sql -- funcao calcular_duracao()
2  CREATE OR REPLACE FUNCTION calcular_duracao()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      IF NEW.Data_Fim IS NOT NULL THEN
6          NEW.Duracao := NEW.Data_Fim - NEW.Data_Inicio;
7      ELSE
8          NEW.Duracao := NULL;
9      END IF;
10     RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 CREATE TRIGGER trg_calc_duracao_atividade
15 BEFORE INSERT OR UPDATE ON ATIVIDADE
16 FOR EACH ROW EXECUTE PROCEDURE calcular_duracao();
```

Código 3: Trigger de cálculo do atributo derivado Duração (N11).

O mesmo arquivo concentra ainda as *triggers* de **disjunção** da especialização de Agente de Conformidade (N8: validar_disjuncao_auditor() e validar_disjuncao_certificador()) que abortam a inserção quando a Atribuição diverge da subclasse) e de **sincronização de identidade** dos papéis sobrepostos de mercado (N5: sincronizar_tipo_originador() e sincronizar_tipo_comprador()), que populam automaticamente TIPO_AGENTE_MERCADO).

6.4 Consultas SQL

Foram implementadas seis consultas (mínimo exigido: cinco), cobrindo múltiplas junções, agregação, subconsultas correlacionadas e não-correlacionadas, funções de janela e divisão relacional. Todos os trechos a seguir encontram-se no arquivo scripts_sql/consultas.sql e são espelhados nas constantes SQL_C1–SQL_C6 de codigo_sistema-aplicacao/app.py, onde o valor literal de filtro é substituído por um *placeholder* %s parametrizado.

6.4.1 C1: Rastreabilidade Completa de Lote (Múltiplas Junções)

Dado o número de série de um Lote, retorna seu histórico completo: a Atividade e o Projeto que o geraram, o Originador, o Auditor responsável, o Certificador que emitiu o Laudo e a transação mais recente. Emprega sete junções, combinando JOIN (relacionamentos obrigatórios) e LEFT JOIN (relacionamentos opcionais, como Projeto, Auditor e Laudo), além de uma CTE que isola a última transação por lote via DISTINCT ON.

```
1 WITH ultima_transacao AS (
```

```

2      SELECT DISTINCT ON (tl.Lote)
3          tl.Lote                AS num_serie,
4          t.Nota_Fiscal,
5          t.Data_Hora            AS data_ultima_transacao,
6          t.Valor                AS valor_ultima_transacao,
7          pj_c.Nome_Fantasia AS comprador_atual
8  FROM TRANSACAO_LOTE tl
9  JOIN TRANSACAO      t      ON tl.Transacao = t.Nota_Fiscal
10 JOIN PESSOA_JURIDICA pj_c ON t.Agente_Mercado_Comprador = pj_c.CNPJ
11 ORDER BY tl.Lote, t.Data_Hora DESC
12 )
13 SELECT
14     l.Num_Serie_Registro, l.Quantidade_de_Credito, l.Status_Ciclo_de_Vida,
15     a.Codigo_Ordem_Servico AS atividade,
16     p.Nome_Projeto,
17     pj_orig.Nome_Fantasia AS originador,
18     pj_aud.Nome_Fantasia AS auditor,
19     ld.Numero_do_Protocolo,
20     pj_cert.Nome_Fantasia AS certificador,
21     ut.Nota_Fiscal        AS ultima_nota_fiscal,
22     ut.comprador_atual
23 FROM LOTE l
24 JOIN ATIVIDADE      a      ON l.Atividade      = a.Codigo_Ordem_Servico
25 JOIN ORIGINADOR     orig   ON a.Originador     = orig.CNPJ
26 JOIN PESSOA_JURIDICA pj_orig ON orig.CNPJ      = pj_orig.CNPJ
27 LEFT JOIN PROJETO   p      ON a.Projeto       = p.Num_Licenca_Ambiental
28 LEFT JOIN AUDITOR   aud    ON a.Auditor       = aud.CNPJ
29 LEFT JOIN PESSOA_JURIDICA pj_aud ON aud.CNPJ    = pj_aud.CNPJ
30 LEFT JOIN LAUDO     ld     ON ld.Atividade    = a.Codigo_Ordem_Servico
31 LEFT JOIN CERTIFICADOR cert ON ld.Certificador = cert.CNPJ
32 LEFT JOIN PESSOA_JURIDICA pj_cert ON cert.CNPJ  = pj_cert.CNPJ
33 LEFT JOIN ultima_transacao ut ON ut.num_serie  = l.Num_Serie_Registro
34 WHERE l.Num_Serie_Registro = 'BR-VCS-2024-0001';

```

Código 4: C1: Rastreabilidade completa de um Lote.

6.4.2 C2: Desempenho de Projetos (Agrupamento e Agregação)

Lista, por Projeto, o total de Lotes emitidos, o total de créditos (tCO₂e), o valor financeiro movimentado (Valor × Quantidade) e o número de transações. Utiliza GROUP BY com as funções COUNT e SUM, e LEFT JOIN para que projetos sem lotes apareçam com valores zerados.

```

1 SELECT
2     p.Num_Licenca_Ambiental,
3     p.Nome_Projeto,
4     COUNT(DISTINCT l.Num_Serie_Registro) AS total_lotes_emitidos,
5     COALESCE(SUM(l.Quantidade_de_Credito), 0) AS total_creditos_tCO2e,
6     COALESCE(SUM(l.Valor * l.Quantidade_de_Credito), 0) AS valor_total_lotes,
7     COUNT(DISTINCT tl.Transacao) AS total_transacoes
8 FROM PROJETO p
9 JOIN ATIVIDADE      a      ON a.Projeto      = p.Num_Licenca_Ambiental
10 LEFT JOIN LOTE      l      ON l.Atividade   = a.Codigo_Ordem_Servico
11 LEFT JOIN TRANSACAO_LOTE tl ON tl.Lote      = l.Num_Serie_Registro
12 GROUP BY p.Num_Licenca_Ambiental, p.Nome_Projeto
13 ORDER BY total_creditos_tCO2e DESC;

```

Código 5: C2: Desempenho de projetos por créditos e valor.

6.4.3 C3: Atividades Finalizadas sem Laudo (Subconsulta Não-Correlacionada)

Identifica atividades já encerradas (Data_Fim no passado) que ainda não possuem nenhum Laudo registrado, um indicador de risco de conformidade. Usa uma subconsulta não-correlacionada com NOT IN sobre a tabela LAUDO.

```
1 SELECT
2     a.Codigo_Ordem_Servico,
3     pj.Nome_Fantasia      AS originador,
4     a.Descricao_Atividade,
5     a.Data_Inicio, a.Data_Fim,
6     a.Duracao            AS duracao_dias,
7     a.Credito_Estimado   AS credito_estimado_tC02e,
8     pj_aud.Nome_Fantasia AS auditor_contratado
9 FROM ATIVIDADE a
10 JOIN ORIGINADOR      orig ON a.Originador = orig.CNPJ
11 JOIN PESSOA_JURIDICA pj   ON orig.CNPJ    = pj.CNPJ
12 LEFT JOIN PESSOA_JURIDICA pj_aud ON a.Auditor = pj_aud.CNPJ
13 WHERE a.Data_Fim IS NOT NULL
14       AND a.Data_Fim < CURRENT_DATE
15       AND a.Codigo_Ordem_Servico NOT IN (SELECT l.Atividade FROM LAUDO l)
16 ORDER BY a.Data_Fim;
```

Código 6: C3: Atividades finalizadas sem laudo de auditoria.

6.4.4 C4: Evolução de Preço de um Lote (Função de Janela)

Para um Lote específico, recupera a série histórica de preços ordenada cronologicamente, calculando a variação absoluta e percentual em relação ao registro anterior. Emprega a função de janela LAG() para acessar o preço da linha anterior sem auto-junção, e NULLIF para evitar divisão por zero.

```
1 SELECT
2     hp.Num_Serie_Registro,
3     pj.Nome_Fantasia      AS originador_do_lote,
4     hp.Data,
5     hp.Preco              AS preco_unitario,
6     hp.Preco - LAG(hp.Preco) OVER w AS variacao_abs,
7     ROUND(
8         (hp.Preco - LAG(hp.Preco) OVER w)
9         / NULLIF(LAG(hp.Preco) OVER w, 0) * 100
10    , 2)                  AS variacao_pct
11 FROM HISTORICO_PRECO hp
12 JOIN LOTE      l   ON hp.Num_Serie_Registro = l.Num_Serie_Registro
13 JOIN ATIVIDADE a   ON l.Atividade          = a.Codigo_Ordem_Servico
14 JOIN ORIGINADOR orig ON a.Originador        = orig.CNPJ
15 JOIN PESSOA_JURIDICA pj ON orig.CNPJ       = pj.CNPJ
16 WHERE hp.Num_Serie_Registro = 'BR-VCS-2024-0001'
17 WINDOW w AS (PARTITION BY hp.Num_Serie_Registro ORDER BY hp.Data)
18 ORDER BY hp.Data;
```

Código 7: C4: Evolução histórica de preço com variação percentual.

6.4.5 C5: Divisão Relacional (Originadores aprovados por TODAS as Certificadoras)

Consulta obrigatória de divisão relacional: encontra os Originadores que possuem Laudos emitidos por *todas* as Certificadoras ativas (`Status = 'Apto'`) do sistema. É implementada pela dupla negação (`NOT EXISTS` aninhado), tradução direta do quantificador universal $\forall$: “não existe certificadora ativa para a qual não exista laudo sobre uma atividade do originador”.

```
1 SELECT
2     o.CNPJ,
3     pj.Nome_Fantasia AS originador,
4     pj.Status
5 FROM ORIGINADOR      o
6 JOIN PESSOA_JURIDICA pj ON o.CNPJ = pj.CNPJ
7 WHERE NOT EXISTS (
8     SELECT 1
9     FROM CERTIFICADOR      c
10    JOIN AGENTE_CONFORMIDADE ac ON c.CNPJ = ac.CNPJ
11    JOIN PESSOA_JURIDICA   pjc ON c.CNPJ = pjc.CNPJ
12    WHERE pjc.Status = 'Apto'
13           AND NOT EXISTS (
14               SELECT 1
15               FROM LAUDO      ld
16               JOIN ATIVIDADE a ON ld.Atividade = a.Codigo_Ordem_Servico
17               WHERE a.Originador = o.CNPJ
18                     AND ld.Certificador = c.CNPJ
19           )
20 )
21 ORDER BY pj.Nome_Fantasia;
```

Código 8: C5: Divisão relacional via `NOT EXISTS` aninhado.

6.4.6 C6: Proprietário Atual de Cada Lote Disponível (Subconsulta Correlacionada)

Para cada Lote “Disponível”, determina o proprietário atual por inferência (conforme a decisão arquitetural da Parte 1): se houver transações, o dono é o comprador da mais recente; caso contrário, é o Originador da Atividade que gerou o Lote (posse inicial, conforme N3). Combina uma CTE com `DISTINCT ON` e `COALESCE` para o *fallback* ao Originador.

```
1 WITH proprietario_por_transacao AS (
2     SELECT DISTINCT ON (t1.Lote)
3         t1.Lote                AS num_serie,
4         t.Agente_Mercado_Comprador AS cnpj_dono_atual,
5         pj.Nome_Fantasia          AS nome_dono_atual,
```

```

6         t.Data_Hora                AS data_aquisicao,
7         'Transacao'                AS origem_posse
8     FROM TRANSACAO_LOTE    t1
9     JOIN TRANSACAO        t  ON t1.Transacao = t.Nota_Fiscal
10    JOIN PESSOA_JURIDICA  pj ON t.Agente_Mercado_Comprador = pj.CNPJ
11    ORDER BY t1.Lote, t.Data_Hora DESC
12 )
13 SELECT
14     l.Num_Serie_Registro,
15     l.Quantidade_de_Credito        AS creditos_tC02e,
16     l.Valor                        AS preco_unitario_atual,
17     COALESCE(pt.cnpj_dono_atual, a.Originador)    AS cnpj_proprietario,
18     COALESCE(pt.nome_dono_atual, pj_orig.Nome_Fantasia) AS proprietario_atual,
19     COALESCE(pt.origem_posse, 'Originador inicial') AS origem_posse
20 FROM LOTE                l
21 JOIN ATIVIDADE          a      ON l.Atividade = a.Codigo_Ordem_Servico
22 JOIN PESSOA_JURIDICA  pj_orig ON a.Originador = pj_orig.CNPJ
23 LEFT JOIN proprietario_por_transacao pt ON pt.num_serie = l.Num_Serie_Registro
24 WHERE l.Status_Ciclo_de_Vida = 'Disponível'
25 ORDER BY l.Num_Serie_Registro;

```

Código 9: C6: Proprietário atual inferido de cada lote disponível.

6.5 A Aplicação (Protótipo Desktop)

A aplicação, desenvolvida em Python na pasta `codigo_sistema-aplicacao/`, é uma interface gráfica desktop (`tkinter`) que se conecta ao PostgreSQL via `psycopg2`. Ela implementa duas funcionalidades de cadastro de dados com tratamento de erros e seis consultas parametrizadas, demonstrando de forma prática o uso da plataforma CarbonTrack.

6.5.1 Execução Parametrizada das Consultas

As consultas selecionadas pelo usuário são executadas com os parâmetros enviados via *placeholders* `%s`, e não por concatenação de *strings*, tornando a aplicação imune a injeção de SQL.

Localização: `codigo_sistema-aplicacao/app.py`, método `CarbonTrackApp.executar_consulta`

```

1 # codigo_sistema-aplicacao/app.py -- CarbonTrackApp.executar_consulta()
2 with self.conn.cursor() as cur:
3     cur.execute(info["sql"], tuple(valores)) # parametros via %s
4     colunas = [d[0] for d in cur.description]
5     linhas = cur.fetchall()

```

Código 10: Execução parametrizada de consulta (proteção contra SQL injection).

6.5.2 Funcionalidades de Inserção

O protótipo disponibiliza formulários para o cadastro de **Projetos** e de **Atividades** de campo (seção 6.3). Antes de submeter a inserção ao banco, a aplicação executa diversas

verificações e tratamentos de erro:

- assegura o preenchimento dos campos obrigatórios e o formato correto de datas e valores monetários (via máscaras de entrada);
- calcula o atributo derivado *Duração* a partir das datas (N10/N11);
- resolve as chaves estrangeiras (Originador, Projeto, Auditor) a partir de listas carregadas do banco, evitando referências inexistentes;
- converte exceções do `psycopg2` (violação de PK, FK, CHECK ou NOT NULL) em mensagens claras ao usuário, no método `_tratar_erro_banco()`.

Figura 3: Tela de cadastro de dados (aba “Cadastro de Dados”).

6.5.3 Funcionalidades de Consulta

A aba de consultas permite selecionar qualquer uma das seis consultas (C1–C6). Para aquelas que exigem entrada (C1 e C4, que recebem o número de série de um Lote), a aplicação gera dinamicamente o campo de parâmetro correspondente; as demais são executadas diretamente. Os resultados são exibidos em uma tabela com rolagem (*Treeview*), prontos para análise e verificação.

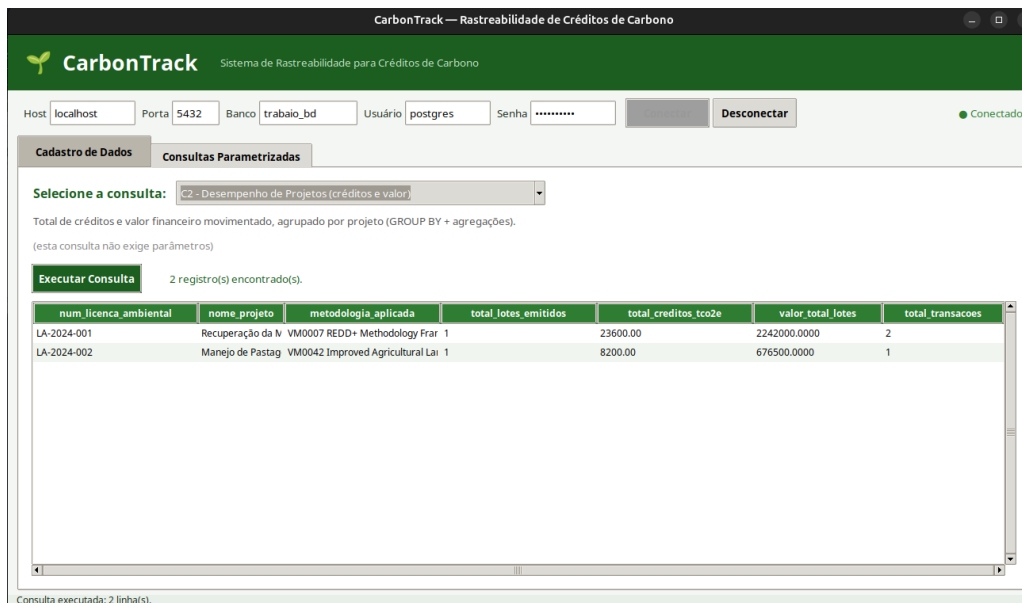


Figura 4: Tela de consultas parametrizadas e exibição de resultados.

7 Conclusão

O desenvolvimento do projeto CarbonTrack proporcionou uma visão integral do ciclo de vida de um sistema de banco de dados, desde a concepção da ideia até a implementação prática de uma aplicação funcional. A construção evoluiu de forma incremental: partiu-se da modelagem conceitual (MER), seguiu-se para o refinamento do modelo relacional, com as devidas correções a partir do *feedback* das entregas anteriores, e culminou na materialização do esquema em PostgreSQL, na codificação das regras de negócio e na criação do protótipo de interface.

Entre os principais desafios encontrados ao longo do projeto, destacam-se o mapeamento das hierarquias de especialização e a inferência de propriedade dos Lotes. Traduzir as especializações (sobreposta para Agente de Mercado e disjunta para Agente de Conformidade) preservando totalidade e disjunção exigiu decisões cuidadosas, visto que o modelo relacional não garante essas restrições nativamente, demandando o uso de *triggers* e validações na camada de aplicação. De forma semelhante, a decisão de inferir o dono atual de um Lote por consulta, em vez de armazená-lo de forma redundante, evitou ciclos e inconsistências no modelo, mas elevou a complexidade das consultas C1 e C6, que combinam CTEs, `DISTINCT ON` e `COALESCE`. A formulação da consulta C5 via dupla negação (`NOT EXISTS` aninhado) foi igualmente exigente, por traduzir o quantificador universal da álgebra relacional para SQL. Por fim, a integração entre aplicação e banco de dados demandou atenção especial no tratamento de erros do SGBD (violações de PK, FK e `CHECK`) e na sincronização entre as validações da interface e as restrições do banco.

O projeto consolidou, na prática, conceitos fundamentais vistos em aula: a tradução

sistemática de um MER para o modelo relacional, as técnicas de mapeamento de relacionamentos e especializações, a escrita de consultas SQL de complexidade crescente (junções múltiplas, agregação, subconsultas correlacionadas, funções de janela e divisão relacional) e o uso de *triggers* em PL/pgSQL para automatizar regras de negócio. O emprego de **Docker** para padronizar o ambiente e de **Git** para o controle de versão evidenciou ainda a importância de boas práticas de engenharia de software no trabalho colaborativo em equipe.

A estrutura do trabalho em etapas progressivas mostrou-se eficaz, pois permitiu amadurecer o modelo gradualmente em vez de concentrar todo o esforço ao final. Como sugestão para turmas seguintes, seria proveitoso que o material de apoio incluísse mais exemplos práticos de mapeamento de especializações e de implementação de *triggers*, que estiveram entre os pontos mais trabalhosos. Antecipar a definição das consultas analíticas para o início do projeto também ajudaria a validar mais cedo se o modelo relacional atende adequadamente aos requisitos de recuperação de dados, evitando ajustes tardios no esquema. Por fim, a adoção de um ambiente containerizado desde a primeira etapa reduziria atritos de configuração entre os membros do grupo.